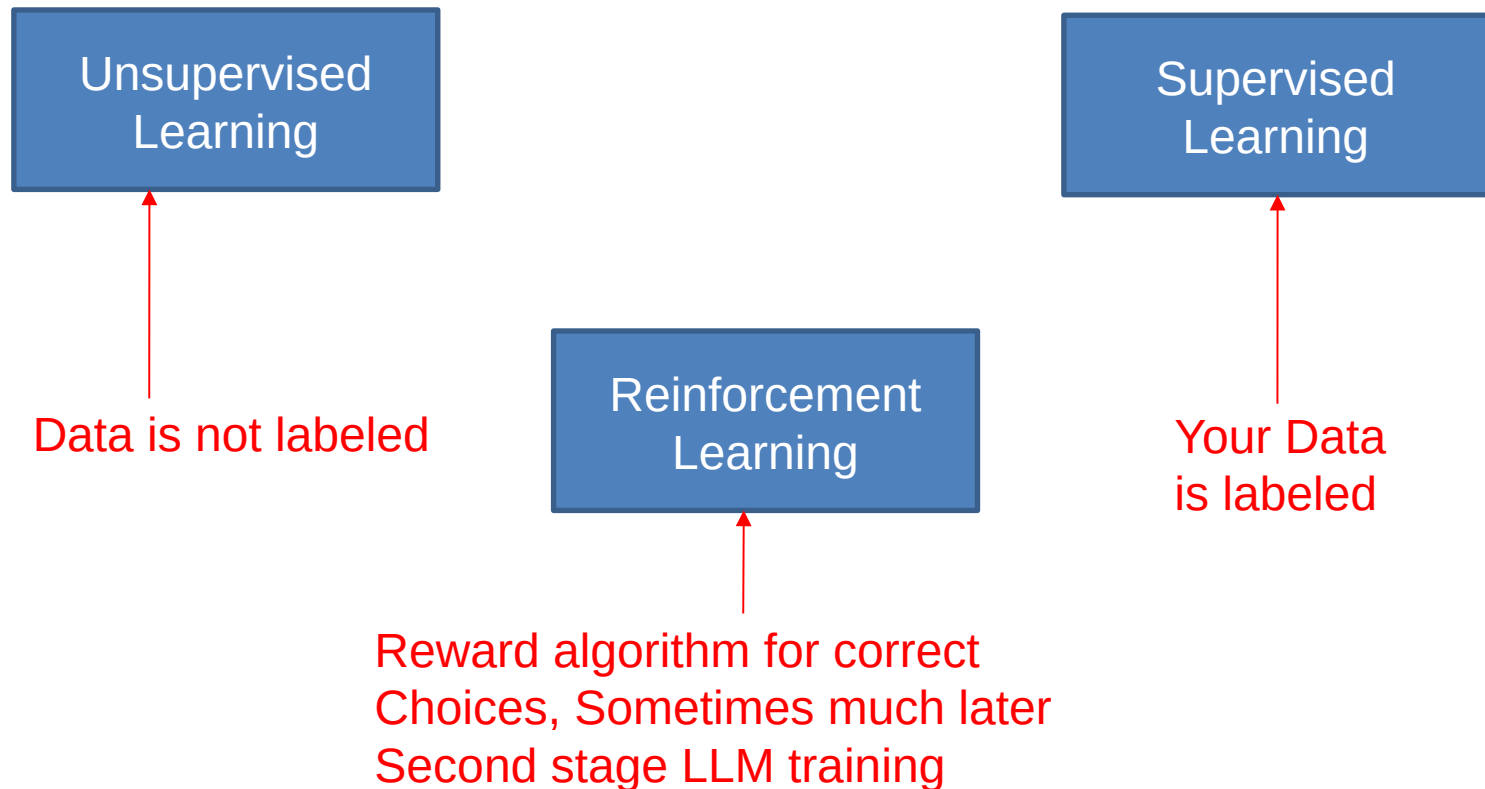
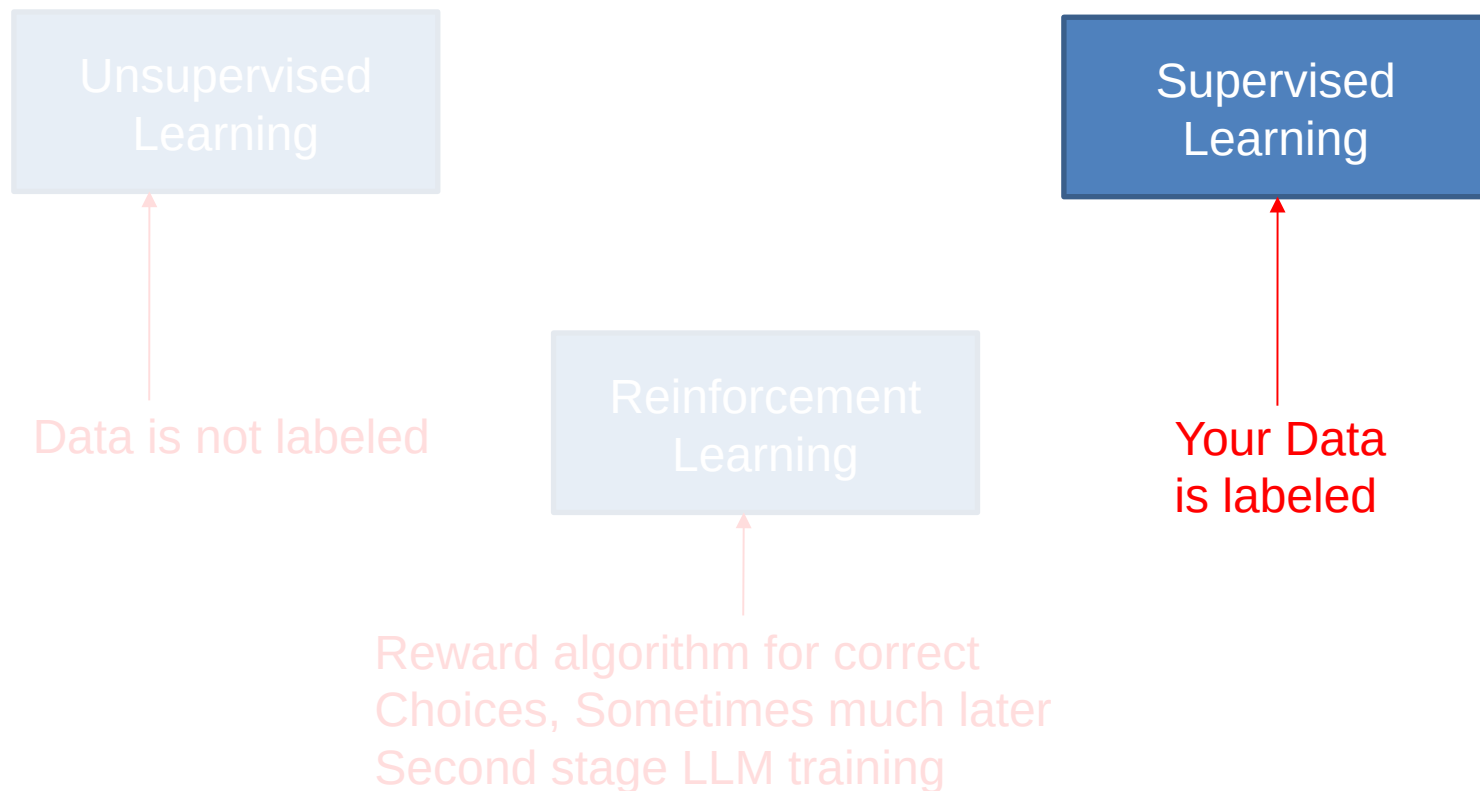


Data:
Supervised Learning, splitting the
dataset

Some types of machine learning



Some types of machine learning



Aside: Semi-Supervised Learning

Have a little labeled data, and a lot of unlabeled data

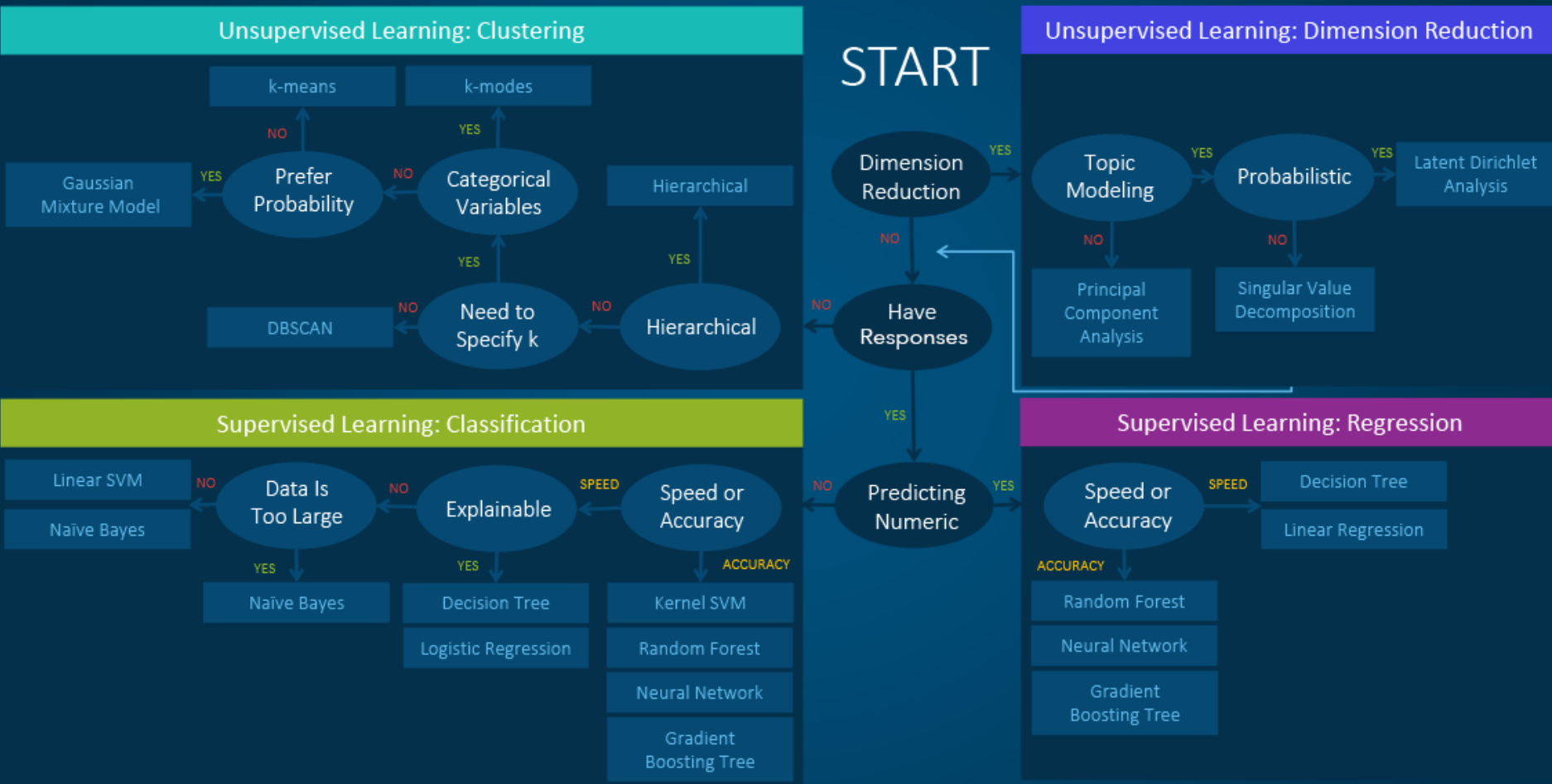
Train a classifier on the labeled data

Use that classifier to predict what unlabeled data is

Add all high confidence ($>80\%$)* predictions to labeled dataset

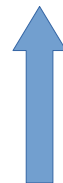
Supervised and Unsupervised Learning

Machine Learning Algorithms Cheat Sheet



Supervised Learning

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa



Known
Target

Supervised

- Labeled Data
- Model knows correct result during training
- Uses correct result to adjust model params during training so its more likely to pick correct result
- Goal is to train model to pick correct result on unseen data
- Examples: Regressions, Decision Trees, Neural Networks

Data

Data: Supervised Setup (easy part)

Still need to do pre-processing (no standardization for tree based methods needed although it does not hurt)

Additionally:

- Need to divide data into 3 portions; train, validation, test
- **What about unbalanced datasets?**
- Typically 80% train 20% test (or 70-30, or 90-10 depends on amount of data you have)
- **Calculate standardization metrics on the train set only. Apply these IDENTICAL metrics to the test set.**
- **If you violate this rule, test set information will leak over to the train set.**

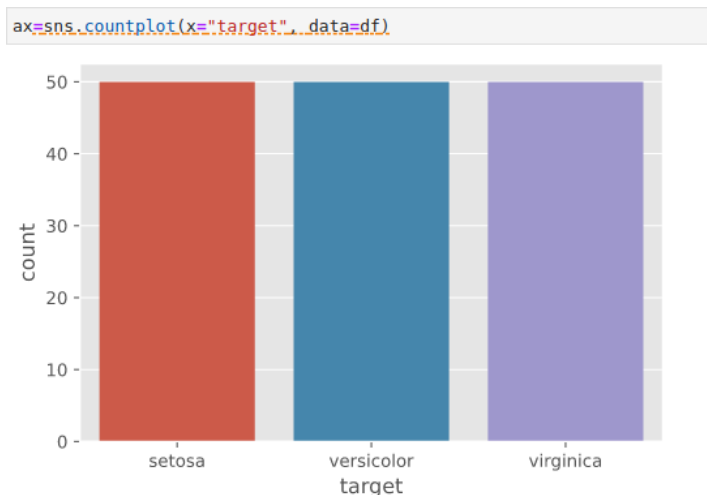
Data: Balanced datasets

If your dataset has roughly the same number of labeled classes then its easy to do a train test split. Here we load the iris dataset, 77% of it is used to train, 33% to test.

```
from sklearn.model_selection import train_test_split
from sklearn import datasets
iris = datasets.load_iris()

X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
```

Notice we have an equal number of iris types



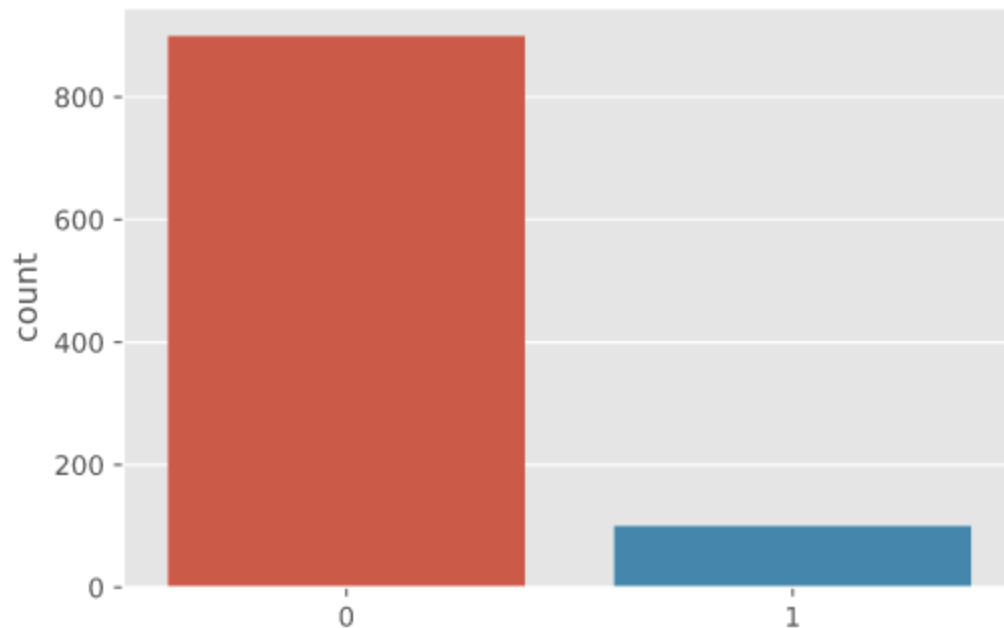
Data: Imbalanced datasets

Dataset has more of one class than others.

This is very common, think of cancer diagnosis

Balance the train portion of the dataset only (not the test set)

```
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=[900, 100], centers=None, cluster_std=[10.0, 2], random_state=22, n_features=2)
ax=sns.countplot(x=y)
```



Data: Imbalanced datasets,

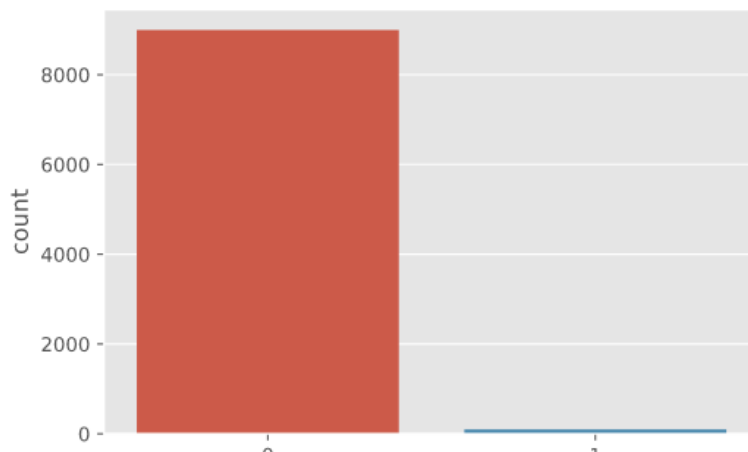
What if dataset is very imbalanced?

For fraud detection, 99.9% of transactions are legitimate.

Which means a fraud dataset consists of almost all legitimate transactions.

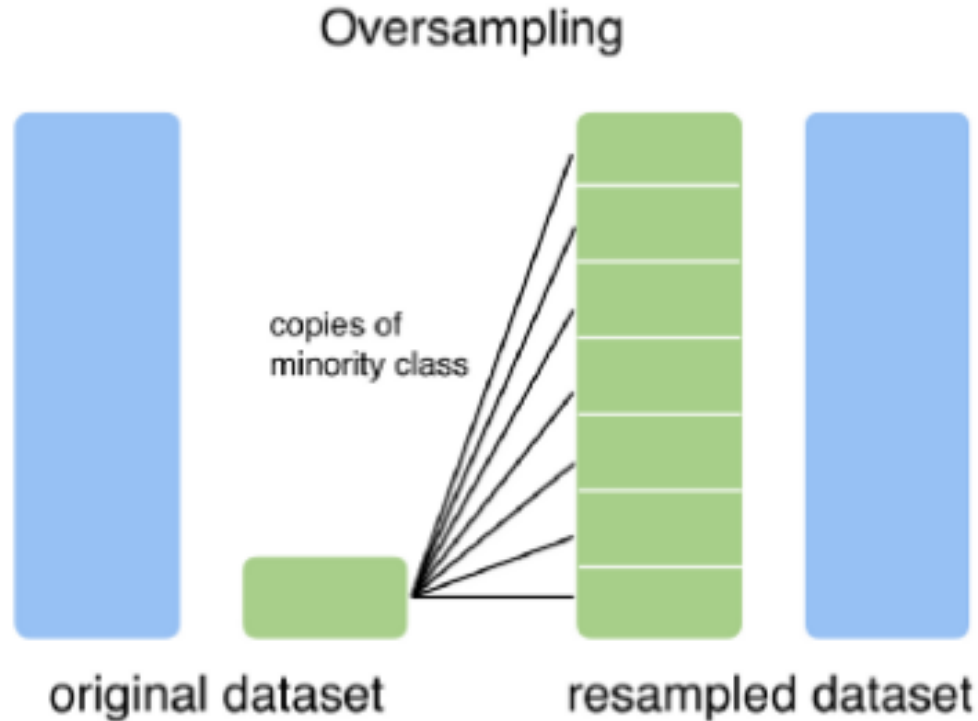
BTW for this case, if your evaluation metric is accuracy, a model may learn to always say its not fraud (99.9% accuracy!).

```
from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=[9000, 100], centers=None, cluster_std=[10.0, 2], random_state=22, n_features=2)
ax=sns.countplot(x=y)
```



Data: Imbalanced datasets, Guides

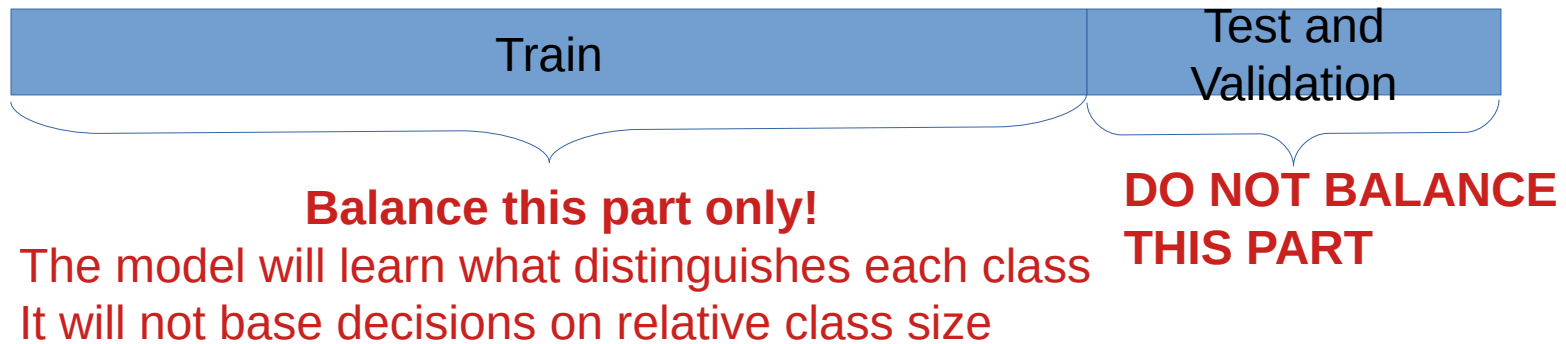
If you don't have a lot of data (<10,000 samples)
oversample the minority (train set only)



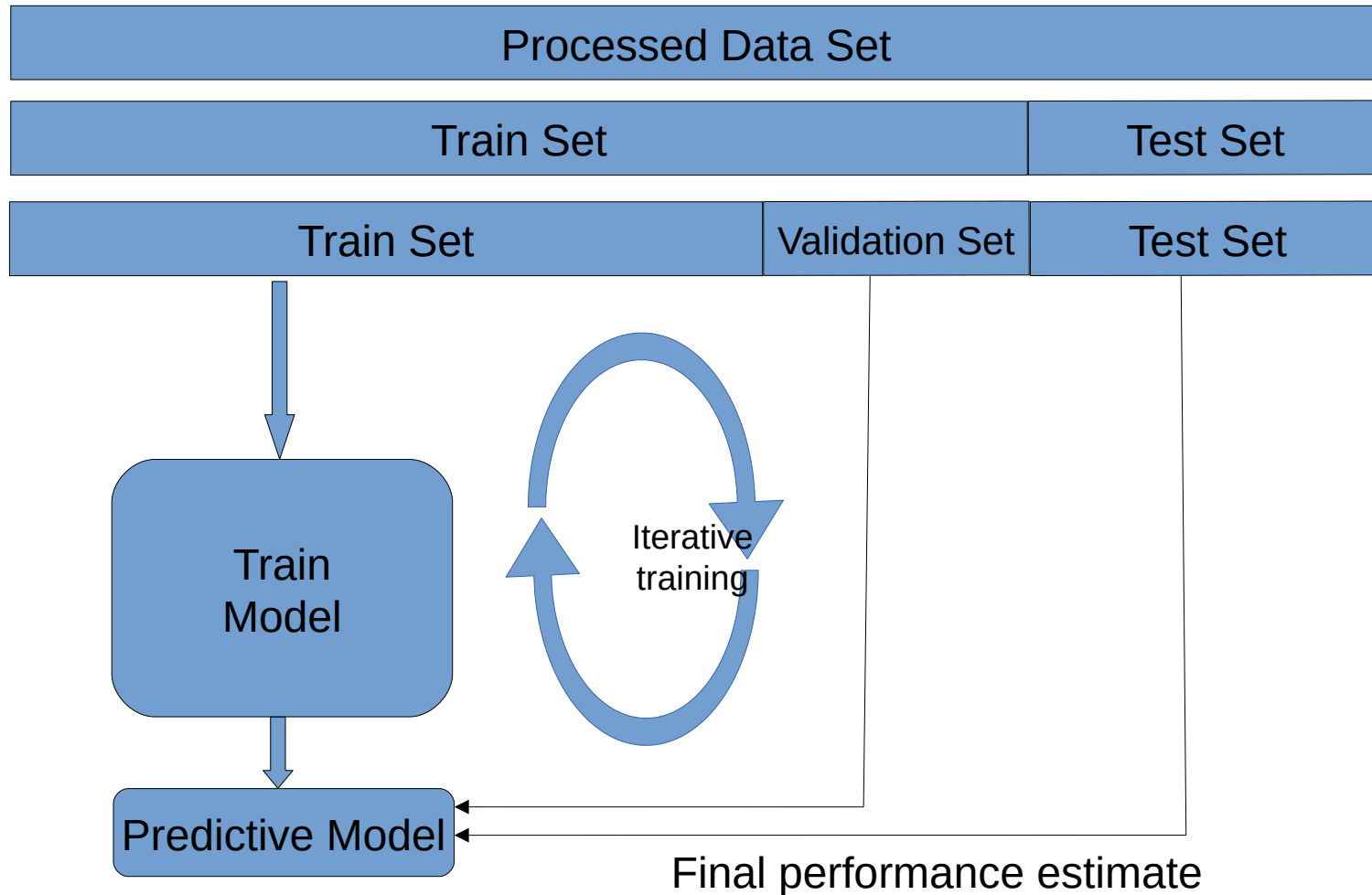
Data: Imbalanced datasets, Guides

Maybe the easiest (or hardest) of all;
Collect more data from the minority class

Data: Imbalanced datasets, Guides



Data: Training a Model



Summary

Three types of machine learning

We are going to do supervised learning

Calculate all metrics on train set only

- Use these metrics to normalize test set(mean, std_dev)

Dealing with unbalanced datasets (balance train set only!)

Summary